EAST WEST UNIVERSITY

CSE430: SOFTWARE TESTING AND QUALITY ASSURANCE
SESSION: SUMMER 2026
SECTION: 01

PROJECT NAME: CAR SHOP MANAGEMENT SYSTEM
PROGRAMMING LANGUAGE: JAVA

SUBMITTED TO:
SHARTAZ SAJID NAHID
LECTURER
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SUBMITTED BY:
TEAM NAME: STACK UNDERFLOW

SUJOY KARMAKAR--------2023-3-60-514
SHAHRIAR AREFIN--------2023-3-60-517
MD. SADMAN----------2023-2-60-304
RUDBATUNNESA----------2021-3-60-006
LIMA PRODHAN----------2021-3-60-180

Car Shop Management System

Testing & Quality Assurance Overview



Project Overview

Application Type

A comprehensive GUI-based desktop application built with **Java** and features a user-friendly interface.

Core Functions

Inventory management, customer tracking, and purchase history.

Key Feature

Seamless integration of car inventory with customer relationship management.

Car Shop Management System

Available Cars

Log Out

Honda Civic (2021)

Make: Honda
Model: Civic
Year: 2021
Price: 2800000 Taka
Fuel Efficiency: 14.5 KMPL
Seating Capacity: 5 persons
Trunk Space: 450.0 cubic feet
Family Friendly: □

Estimate Insurance | Calculate Trip Cost | Purchase

Porsche 911 (2022)

Make: Porsche
Model: 911
Year: 2022
Price: 15000000 Taka
Horsepower: 450 HP
Top Speed: 310 KM/H
0-60 KM/H Acceleration: 3.2 seconds
Is Hyper Car: □
Track Ready: □

Estimate Insurance | Maintenance Cost | 1/4 Mile Time | Purchase

Ferrari F8 Tributo (2023)

Make: Ferrari
Model: F8 Tributo
Year: 2023
Price: 35000000 Taka
Horsepower: 710 HP
Top Speed: 340 KM/H

Switch to Admin Panel

Car Shop Management System

Admin Dashboard

Add Commuter Car | Add Sports Car

Car Make:

Car Model:

Car Year:

Car Price (Taka):

Fuel Efficiency (KMPL):

Seating Capacity:

Trunk Space (Liters):

Add Commuter Car

Shop Statistics:

Total Inventory Value: 52800000 Taka
Average Car Price: 17600000.00 Taka
Most Expensive Car: Ferrari F8 Tributo (35000000 Taka)
Cheapest Car: Honda Civic (2800000 Taka)

Current Inventory:

Make: Honda | Model: Civic | Year: 2021 | Price: 2800000 Taka | Fuel Efficiency: 14.5 K

Make: Porsche | Model: 911 | Year: 2022 | Price: 15000000 Taka | Horsepower: 450 H

Make: Ferrari | Model: F8 Tributo | Year: 2023 | Price: 35000000 Taka | Horsepower: 7

Global User Logs:

--- CUSTOMER LOG ---
Name: Shahriar

Address: Aftabnagar

Contact Info: 10345678910

VIP Status: No

Total Spent: 2500000 Taka

Switch to User Panel

Testing Strategy

An overview of the frameworks, methodologies, and coverage goals guiding quality assurance for the Car Shop Management System.



Testing Framework & Approach

CarShopTest.java

```java
public class CarShopTest {

    @BeforeEach
    public void setUp() {
        shop = new CarShop();
        commuter1 = new Commuter( makeParam: "Toyota", modelParam: "Corolla", yearParam: 2020, priceParam: 2500000, fuelEfficiencyParam: 15.0, seatingCapacityParam: 5, trunkSpaceParam: 470.0);
        commuter2 = new Commuter( makeParam: "Honda", modelParam: "Civic", yearParam: 2021, priceParam: 2800000, fuelEfficiencyParam: 14.5, seatingCapacityParam: 5, trunkSpaceParam: 450.0);
        sports1 = new Sports( makeParam: "Porsche", modelParam: "911", yearParam: 2022, priceParam: 15000000, horsepowerParam: 450, topSpeedParam: 310, acceleration0to60Param: 3.2);

        shop.addCarToInventory(commuter1);
        shop.addCarToInventory(commuter2);
        shop.addCarToInventory(sports1);
    }

    @Test
    public void testGetTotalInventoryValue() {
        int expectedTotal = 2500000 + 2800000 + 15000000;
        assertEquals(expectedTotal, shop.getTotalInventoryValue(), message: "Total inventory value should match sum of all cars.");
    }

    @Test
    public void testFindCarsByMake() {
        ArrayList<Car> foundCars = shop.findCarsByMake("Honda");
        assertEquals( expected: 1, foundCars.size(), message: "Should find exactly 1 Honda.");
        assertEquals( expected: "Honda", foundCars.get(0).getMake(), message: "The found car make should be Honda.");

        ArrayList<Car> noCars = shop.findCarsByMake("Ferrari");
        assertTrue(noCars.isEmpty(), message: "Should return empty list for make not in inventory.");
    }
```

Coverage — CarShopTest

Element	Class, %	Method, %	Line, %	Branch, %
carShop.inventory	100% (2/2)	100% (22/22)	98% (79/80)	95% (23/24)
CarShop	100% (1/1)	100% (9/9)	96% (28/29)	95% (23/24)
CarShopTest	100% (1/1)	100% (13/13)	100% (51/51)	100% (0/0)

Framework

JUnit 4 serves as the backbone for all automated testing across the system.

Testing Methodology

- **Unit Testing:** To test individual methods in isolation
- **Parameterized Testing:** To run tests multiple times with different inputs.

Inventory Management Testing (CarShopTest)

Focus: Core inventory management logic



Core Operations

Validates add, remove, and filter vehicle operations across the full inventory lifecycle.



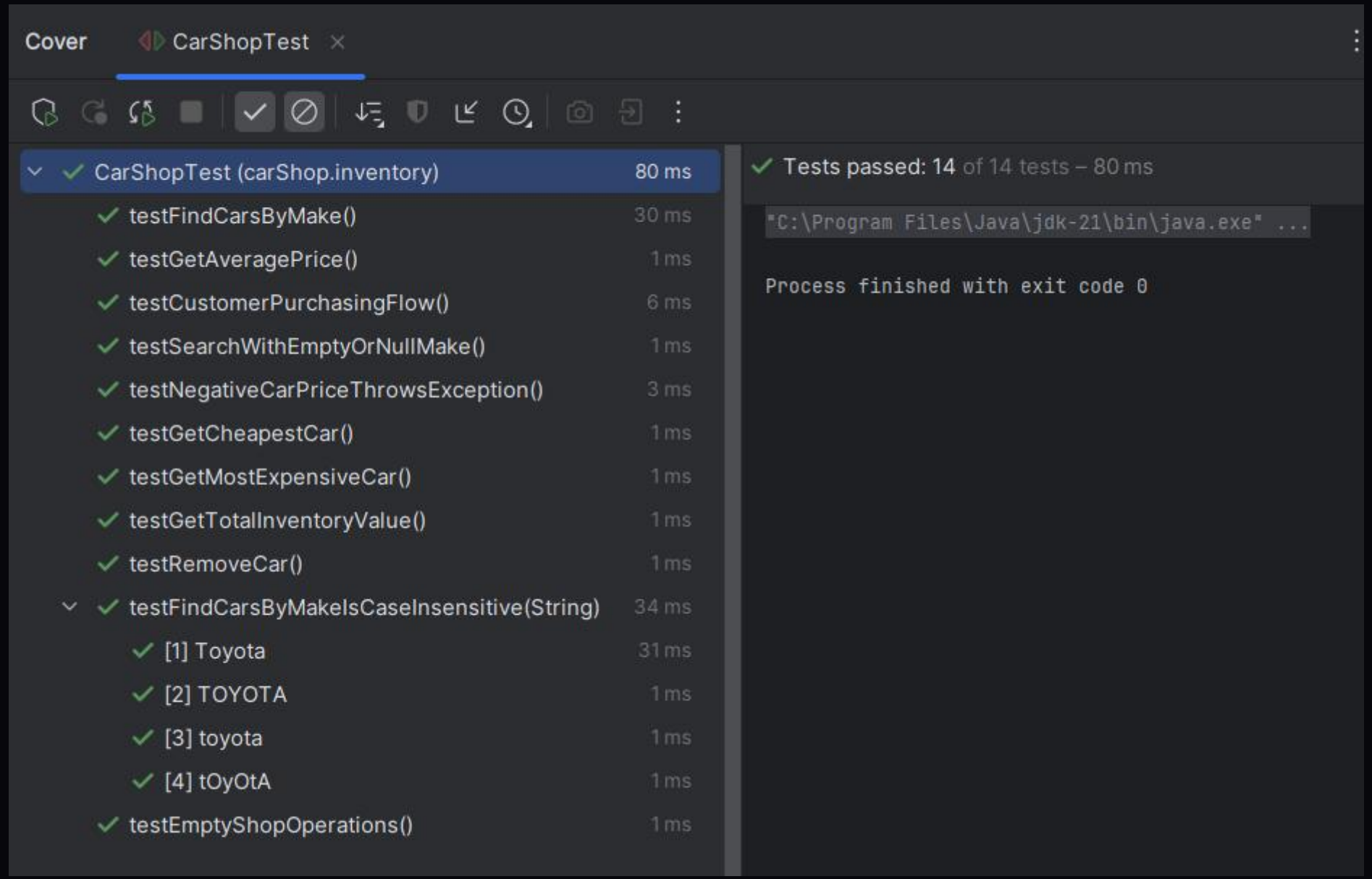
Search Mechanisms

Tests for finding the most expensive, cheapest, and vehicles searched by make.



Validation & Exception Handling

Covers invalid operations such as negative prices and out-of-bounds inputs.


Cover
CarShopTest
CarShopTest (carShop.inventory) 80 ms
testFindCarsByMake() 30 ms
testGetAveragePrice() 1 ms
testCustomerPurchasingFlow() 6 ms
testSearchWithEmptyOrNullMake() 1 ms
testNegativeCarPriceThrowsException() 3 ms
testGetCheapestCar() 1 ms
testGetMostExpensiveCar() 1 ms
testGetTotalInventoryValue() 1 ms
testRemoveCar() 1 ms
testFindCarsByMakeIsCaseInsensitive(String) 34 ms
[1] Toyota 31 ms
[2] TOYOTA 1 ms
[3] toyota 1 ms
[4] tOyOtA 1 ms
testEmptyShopOperations() 1 ms
Tests passed: 14 of 14 tests – 80 ms
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Process finished with exit code 0

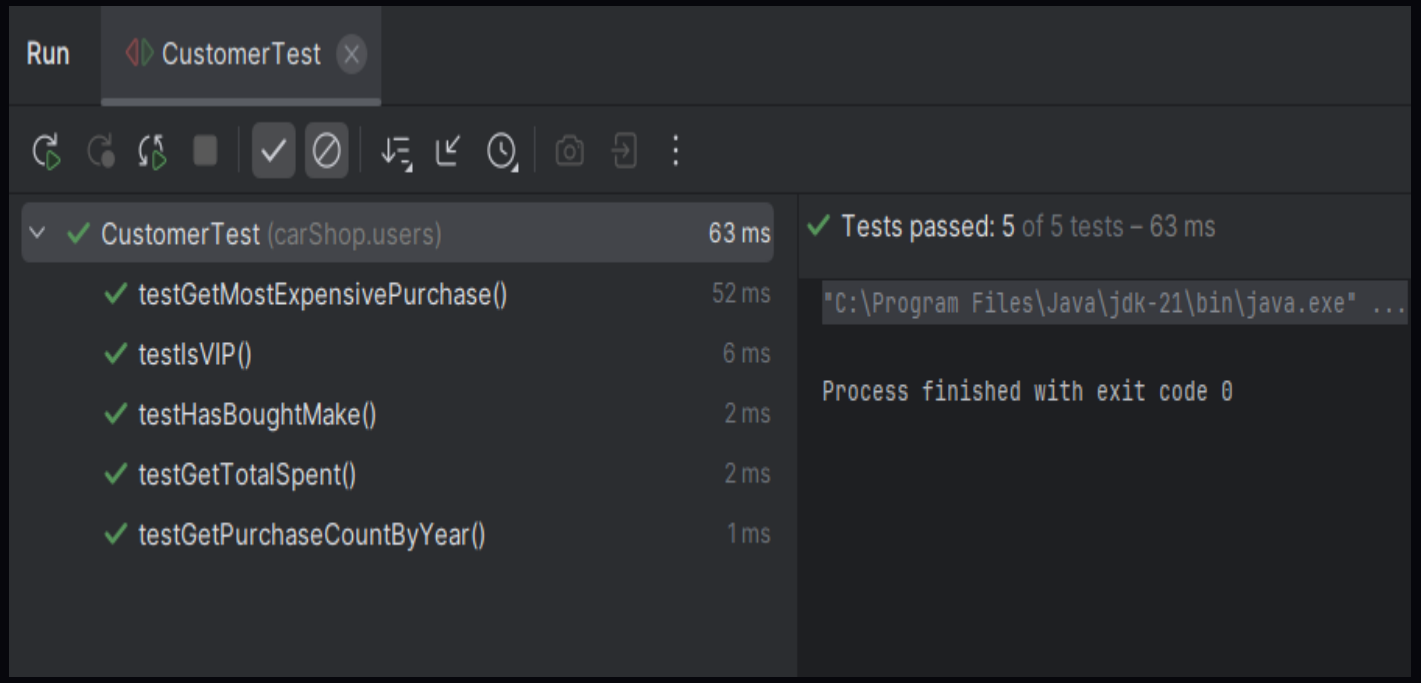
Customer Management Testing(CustomerTest)

Focus

User-centric logic and customer-related operations

CustomerTest.java Highlights

- Verified total spent calculations and VIP status designation.
- Tested tracking logic for identifying the most expensive purchase.
- Validated historical purchase patterns (e.g., counting purchases by year).



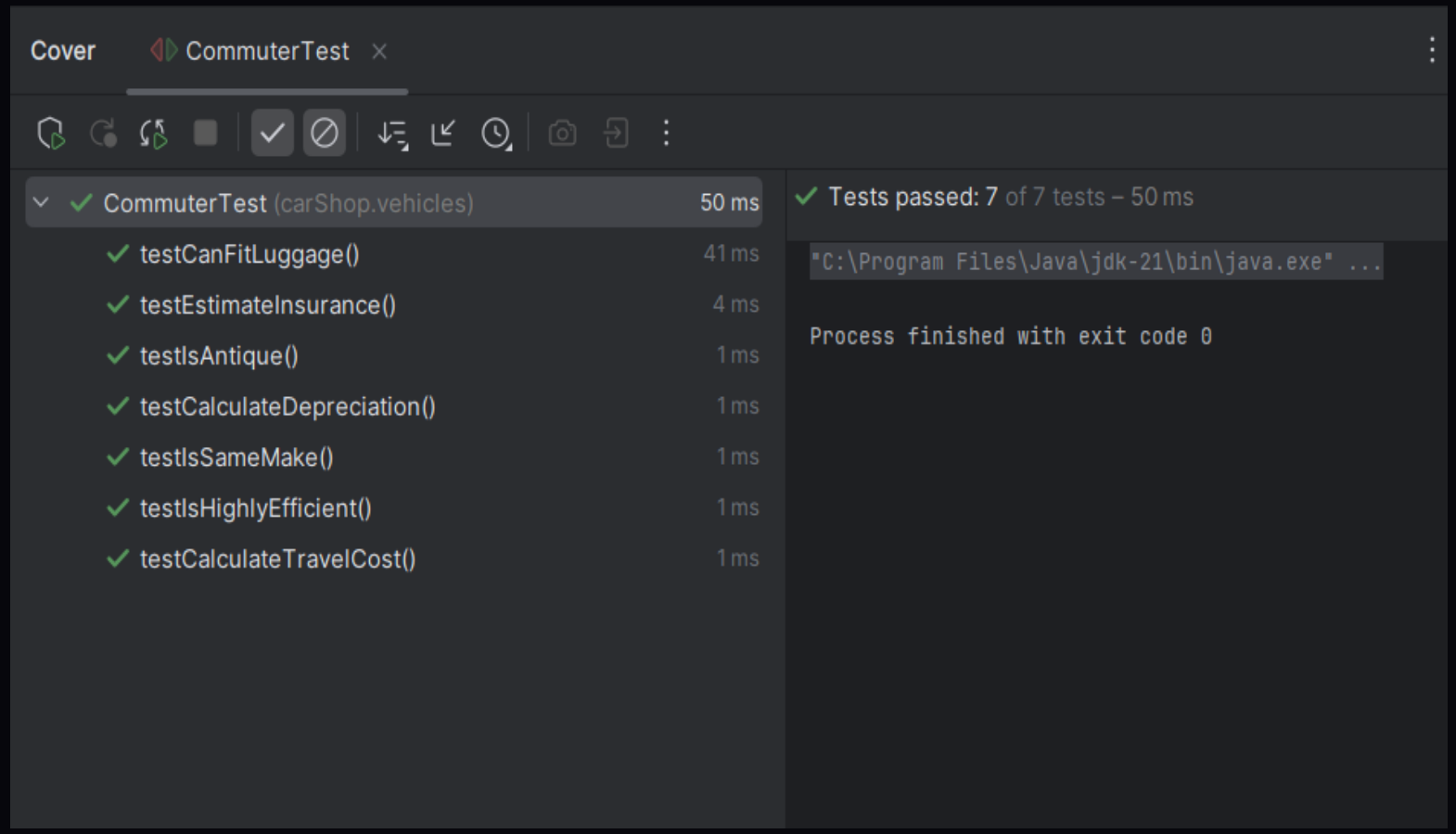
Commuter Vehicle Testing(CommuterTest)

Focus

Business logic and specifications for commuter vehicles.

CommuterTest.java Highlights

- Tested financial estimators: depreciation, insurance, and travel costs.
- Evaluated practical classifications: luggage capacity and efficiency.
- Verified the logic for identifying antique vehicles based on the manufacturing year.



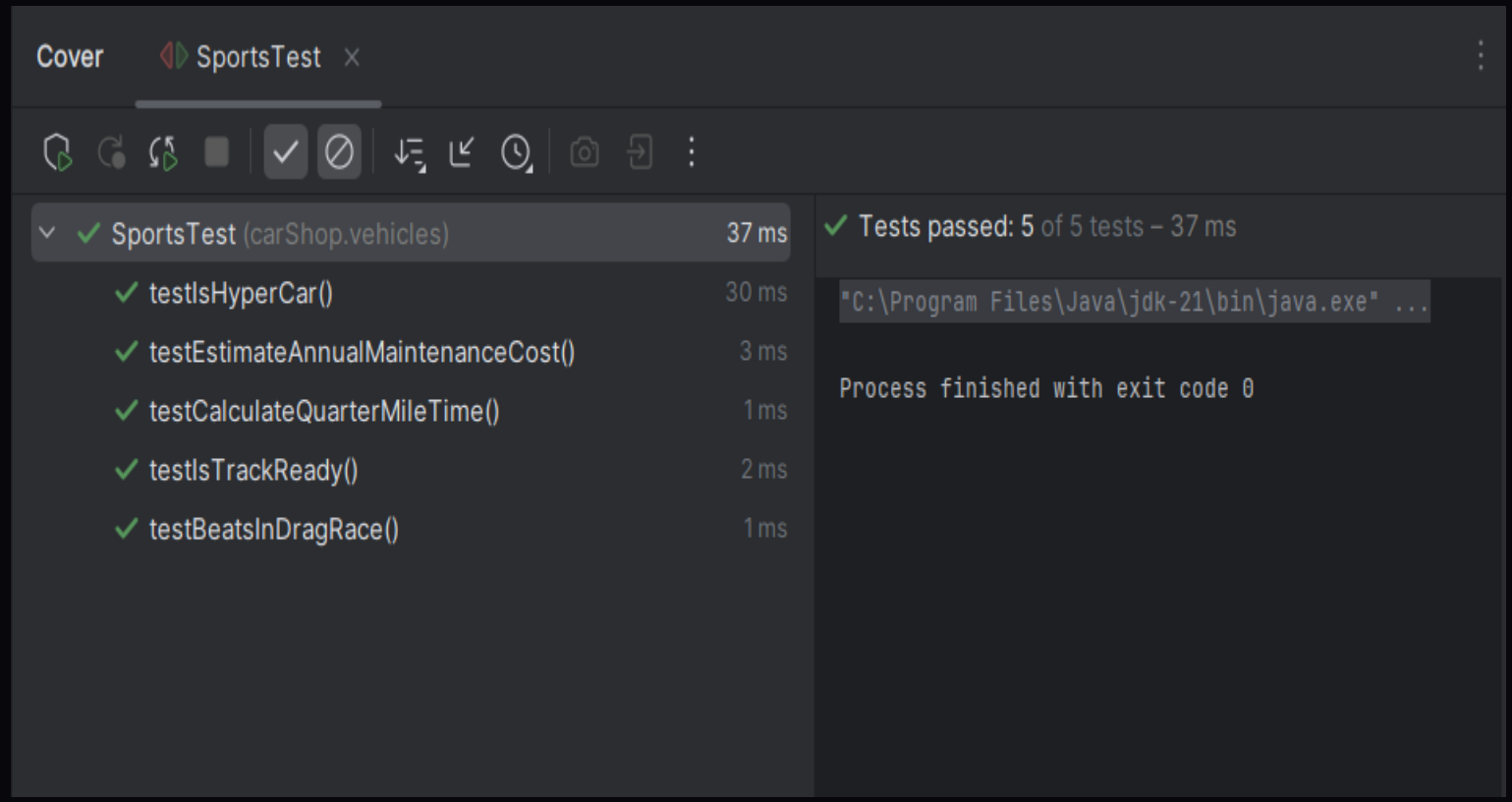
High-Performance Vehicle Testing(SportsTest)

Focus

Distinct attributes of sports and high-performance cars.

SportsTest.java Highlights

- Validated performance categorizations (e.g., 'Is Track Ready', 'Is Hypercar').
- Tested specific metric estimators like quarter-mile time calculations.
- Evaluated comparative logic (e.g., determining drag race winners) and high annual maintenance costs.



High-Performance Vehicle Testing(SportsTest)

TC ID	Test Method	Test Scenario	Input / Test Data	Expected Result	Testing Concept
SP-01	`testIsHyperCar()`	Check whether a normal high-performance car is classified as a hypercar	Porsche 911: 310 km/h, 3.2 sec	`False`	Negative testing; logical condition testing
SP-02	`testIsHyperCar()`	Check whether a qualifying car is classified as a hypercar	Bugatti Chiron: 420 km/h, 2.4 sec	`True`	Positive testing; logical condition testing
SP-03	`testCalculateQuarterMileTime()`	Verify quarter-mile time calculation	Porsche 911: 450 HP, 3.2 sec	≈ **9.42 sec**	Functional/calculation testing; floating-point tolerance
SP-04	`testIsTrackReady()`	Check a car that satisfies track-ready requirements	Porsche 911: 450 HP, 310 km/h	`True`	Positive testing; condition testing
SP-05	`testIsTrackReady()`	Check a car that does not satisfy track-ready requirements	Mazda Miata: 181 HP, 220 km/h	`False`	Negative testing; condition testing
SP-06	`testEstimateAnnualMaintenanceCost()`	Test maintenance calculation below 500 HP	Porsche 911: 450 HP, 10,000 miles	**6000.0**	Branch testing

Thank You!